온라인 광고 로그 처리 파이프라인 및 분석 플랫폼

Team - CAPA (캐파) | Cloud-native AI Pipeline for Ad-logs

ASAC AI 활용 데이터 엔지니어 과정 1기 기업 연계 프로젝트

CAPA

맨토: 정의근 / 팀원: 장현우 · 김시현 · 김병훈

팀 구성 및 역할 분담

장현우

인프라 & AI 엔진

- AWS 클라우드 인프라 아키텍처 (Terraform) 설계
- 자연어 질의(Text-to-SQL) 엔진 및 RAG 시스템 구축

김시현

데이터 수집 & 시각화

- 실시간 데이터 수집 파이프라인 (Kinesis) 구현
- Redash 시각화 대시보드 및 데이터 마킹 설계

김병훈

자동화 리포트 & 이상 탐지

- Airflow 기반 리포트 생성 자동화 (DAG) 개발
- Prophet 모델 기반 로그 이상치 탐지 시스템 구현

목차

CONTENTS

01
프로젝트 배경

기획 배경
문제 원인
도메인 선정
솔루션 제안

02
시연 영상

실시간 자연어 데이터 조회
대시보드
자동화 리포트
광고 이상 탐지 알림

03
전체 프로세스

시스템 전체 아키텍처
데이터 수집 및 저장
자연어 질의 파이프라인
대시보드 구성
리포트 자동화
이상 탐지 알림
인프라 운영
기술 스택

04
결론 및 회고

성과 비교 (AS-IS vs TO-BE)
프로젝트 시행착오 및 회고
향후 발전 방향 및 Q&A

CAPA

SECTION 01

프로젝트 배경

기획 배경

문제 원인

도메인 선정

솔루션 제안

Introduction

문제: 데이터 격차(Data Gap)

데이터가 필요한 사람

마케터 / 기획자 / 운영팀

- ✕ SQL 지식 부족
- ✕ 실시간 분석 불가

≠

접근할 수 있는 사람

개발자 / 데이터 엔지니어

- ✓ SQL 활용 능력 보유
- 분석 요청 병목 발생

⇄

데이터 요구와 실행 사이의 거대한 병목 현상

누구나 공감하는 '기다림'

나 (마케터) 오전 10:03

안녕하세요, 지난달 데이터 좀 확인할 수 있을까요? 🙏

데이터 담당자 오전 10:47

확인해볼게요!

데이터 담당자 오후 3:12

지금 다른 건 처리 중이라 내일 오전 중으로 드릴게요

나 (마케터) 오후 3:13

아 넵... 감사합니다 😅

오전 10:03 → 오후 3:13 = 5시간 10분 경과

"데이터는 어딘가에 있습니다. 근데 꺼낼 수가 없습니다."

격차가 만드는 막대한 손실

💬 커뮤니케이션 비용

요청 → 오해 → 재요청의 반복으로 인한 에너지 낭비

□ 의사결정 골든타임

데이터 팀 백로그로 인한 실시간 시장 대응 실패

🚀 인적 리소스 낭비

전문가들이 단순 데이터 추출에 시간의 40% 이상 사용

$6.2B

생산성 손실 가치

전 세계 기업들이 데이터 대기 시간으로 인해 잃고 있는 연간 추산 비용

왜 이 문제가 구조적으로 생기나?

원인 1 — 기술 장벽

- 데이터를 꺼내려면 SQL을 알아야 함
- 이 장벽 탓에 기업 데이터의 **68%**는 비개발 직군에게 닿지 못하고 방치

출처: Seagate & IDC Report 2020

원인 2 — 분석팀 병목

- 모든 요청이 소수의 분석팀에게 집중
- 진짜 병목 = "데이터 요청 vs 소수의 분석팀"

출처: Instawork: Analytics in the Age of AI

왜 하필 광고 도메인인가?

광고 로그는 실시간 발생으로 발생합니다. 빠르게 보고 즉각 예산을 조정해야 수익이 납니다.

- 하루 30만 건 데이터 생성
- CTR · CVR · ROAS 수치화
- ₩ 1% 효율 개선 = 수억 원 절감

핵심 KPI 산식

- CTR = 클릭 수 / 노출 수
- CVR = 전환 수 / 클릭 수
- ROAS = 매출 / 광고비

상황	고통 (Pain Point)	
일정 주기로 리포트를 만들어야 한다	수기 엑셀 집계	→ 반복 작업에 수 시간 낭비
데이터 분석이 필요하다	SQL 모르면 데이터팀 요청	→ 수 시간 대기
늘 봐야 하는 지표가 있다	매번 요청 반복	→ 리소스 낭비
내가 안 보는 사이 로그가 이상해졌다	뒤늦게 발견	→ 예산 이미 낭비됨

"CAPA는 이 4가지를 각각 해결했습니다."

CAPA가 제시하는 해결책 (4가지 핵심 기능)

Report

Airflow 스케줄링 자동 생성 및 전송

해결: 수기 엑셀 반복 작업 제거

Ask

SQL을 모르더라도 Slack에서 자연어로 즉시 조회

해결: 광고 데이터 즉시 접근 → 의사결정 지연 제거

Dashboard

SQL 없이 Redash 대시보드 원클릭

해결: 반복 요청 없이 언제든 직접 확인

Alert

Impression/Click/Conversion
데이터 실시간 이상 감지

해결: 뒤늦은 발견 → 즉시 알림으로 예산 낭비 방지

"말보다 직접 보는 게 빠릅니다. 지금 바로 시연해드리겠습니다."

SECTION 02

시연 영상

자동화 리포트

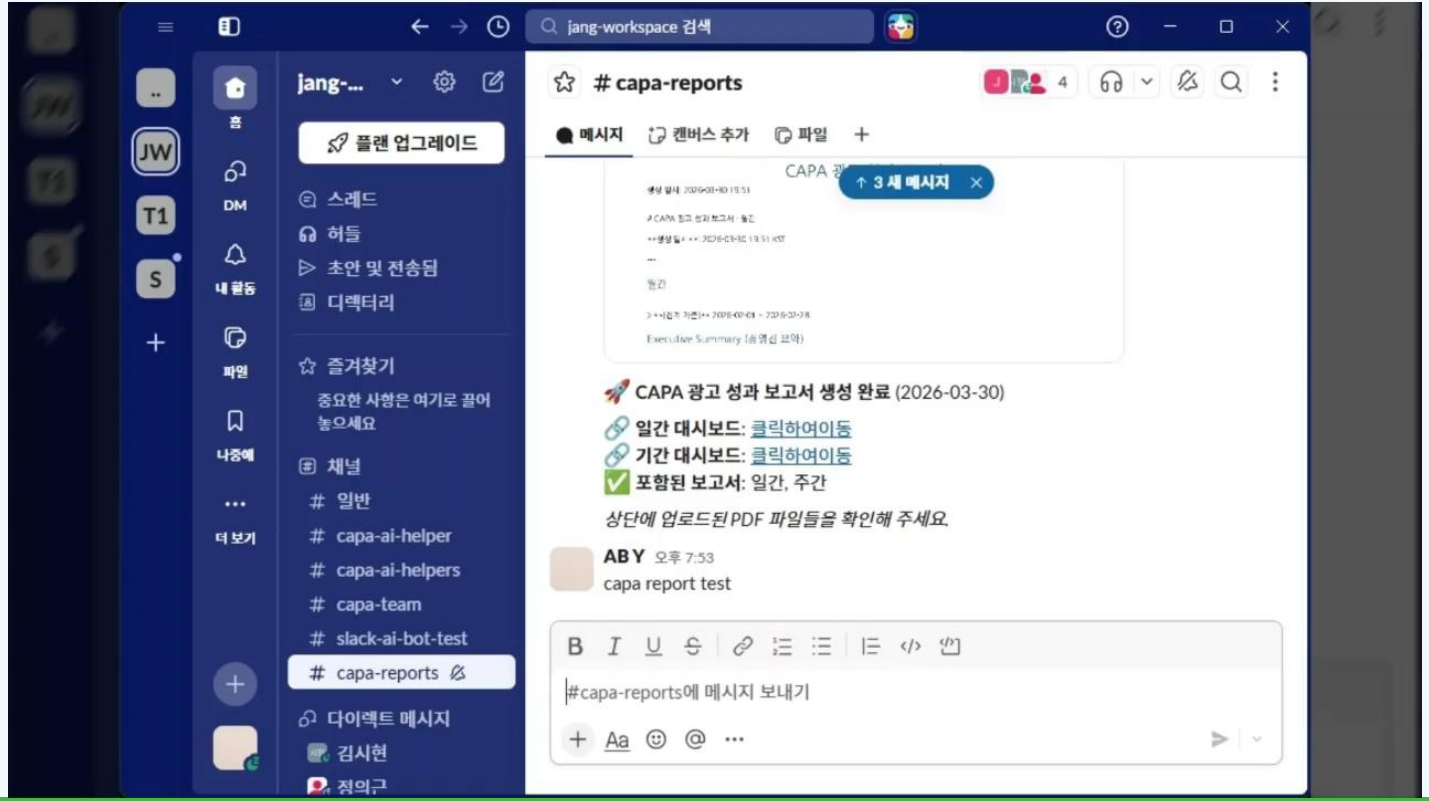
대시보드

자연어 기반
데이터 분석

실시간 이상치
탐지 알림

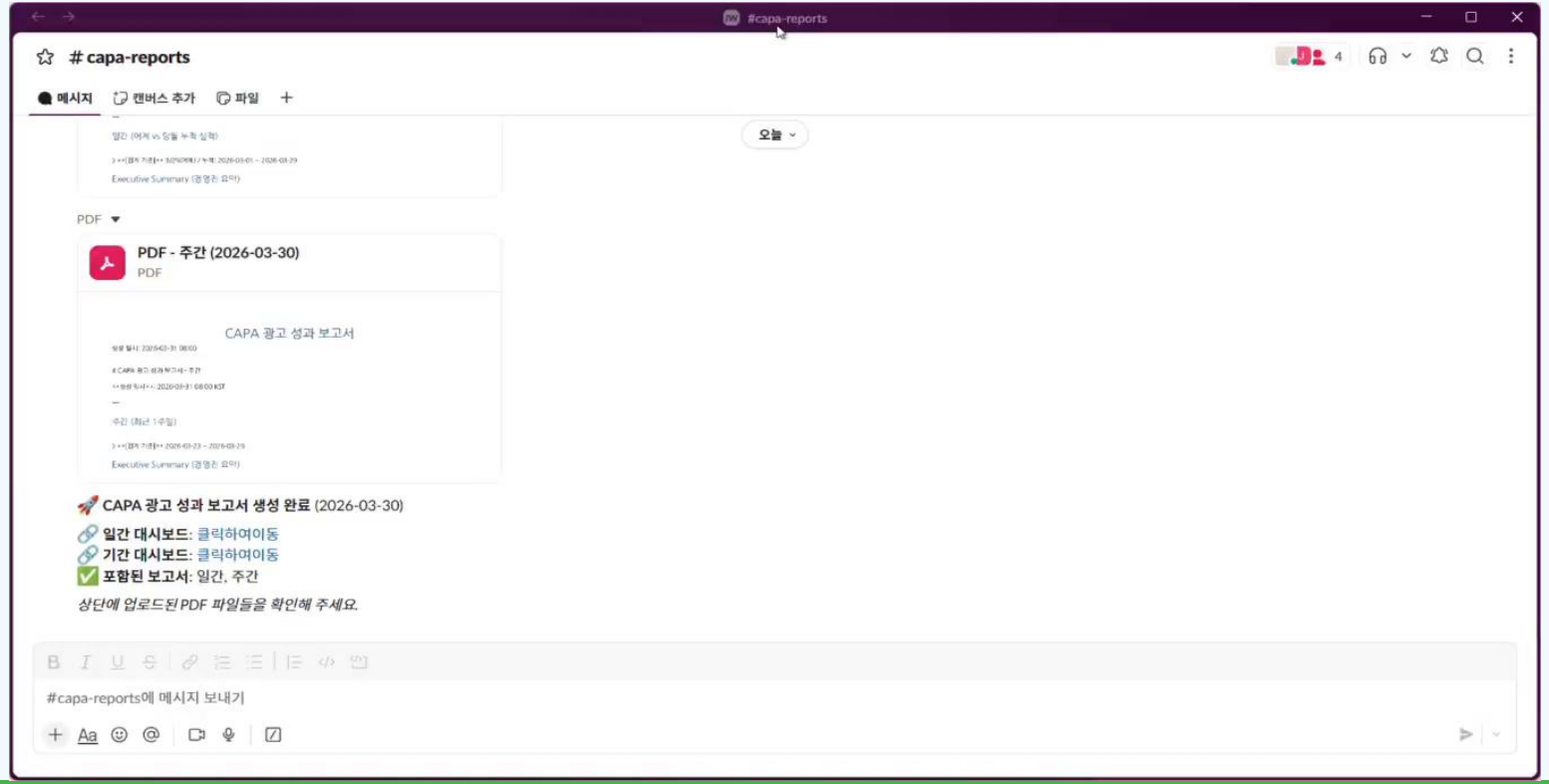
Demo

시연 Scene 1 — 리포트 자동 전송



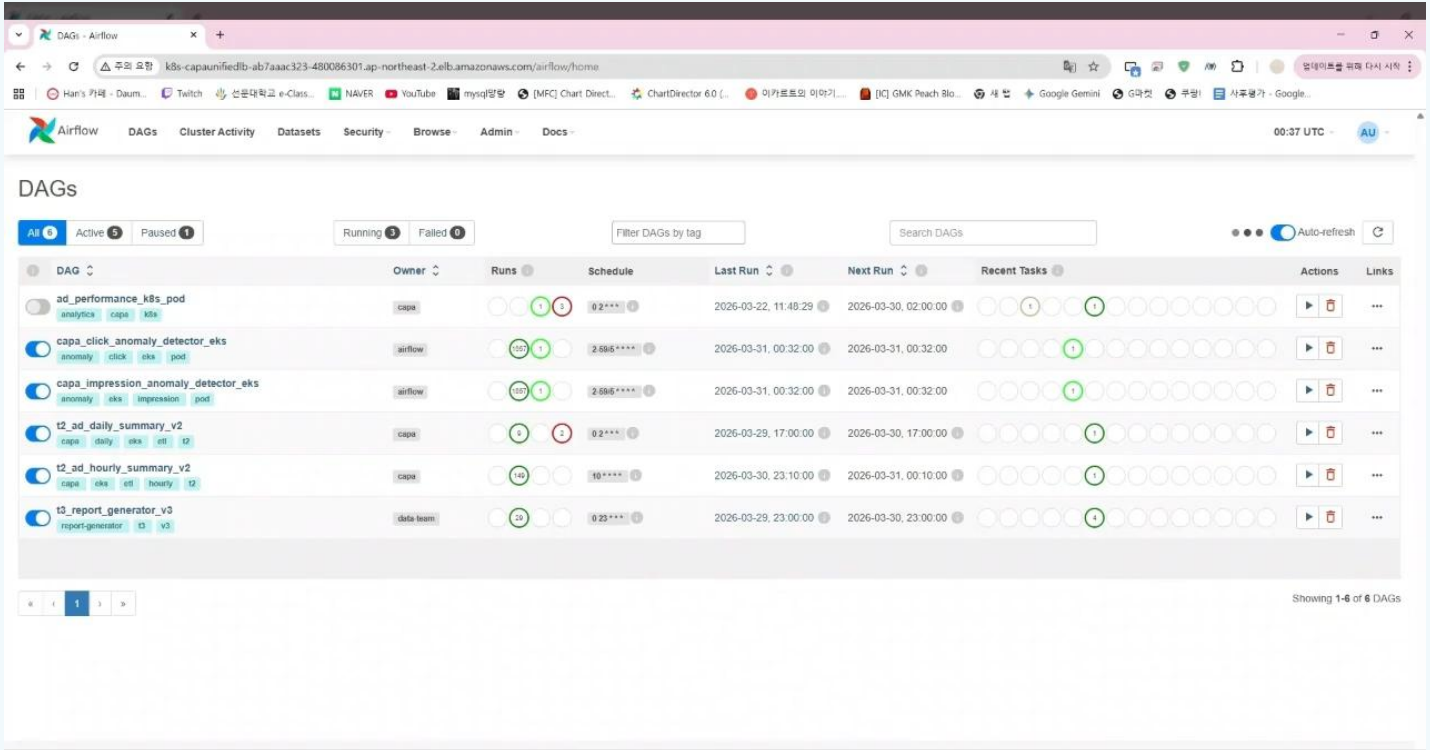
"SQL 지식 없이 매일 아침마다 리포트"

시연 Scene 2 — Redash 대시보드



"SQL 지식 없이 리포트와 함께 대시보드까지 원클릭"

시연 Scene 4 — 이상치 알림



"내가 안 보는 사이에도 이상 감지 → 엔지니어 개입 없이 즉시 마케터에게 알림"

시연 Scene 3 — 자연어 질의(Text-to-SQL)

⚡ 반나절 이상 걸리던 작업을 단 수십 초 만에

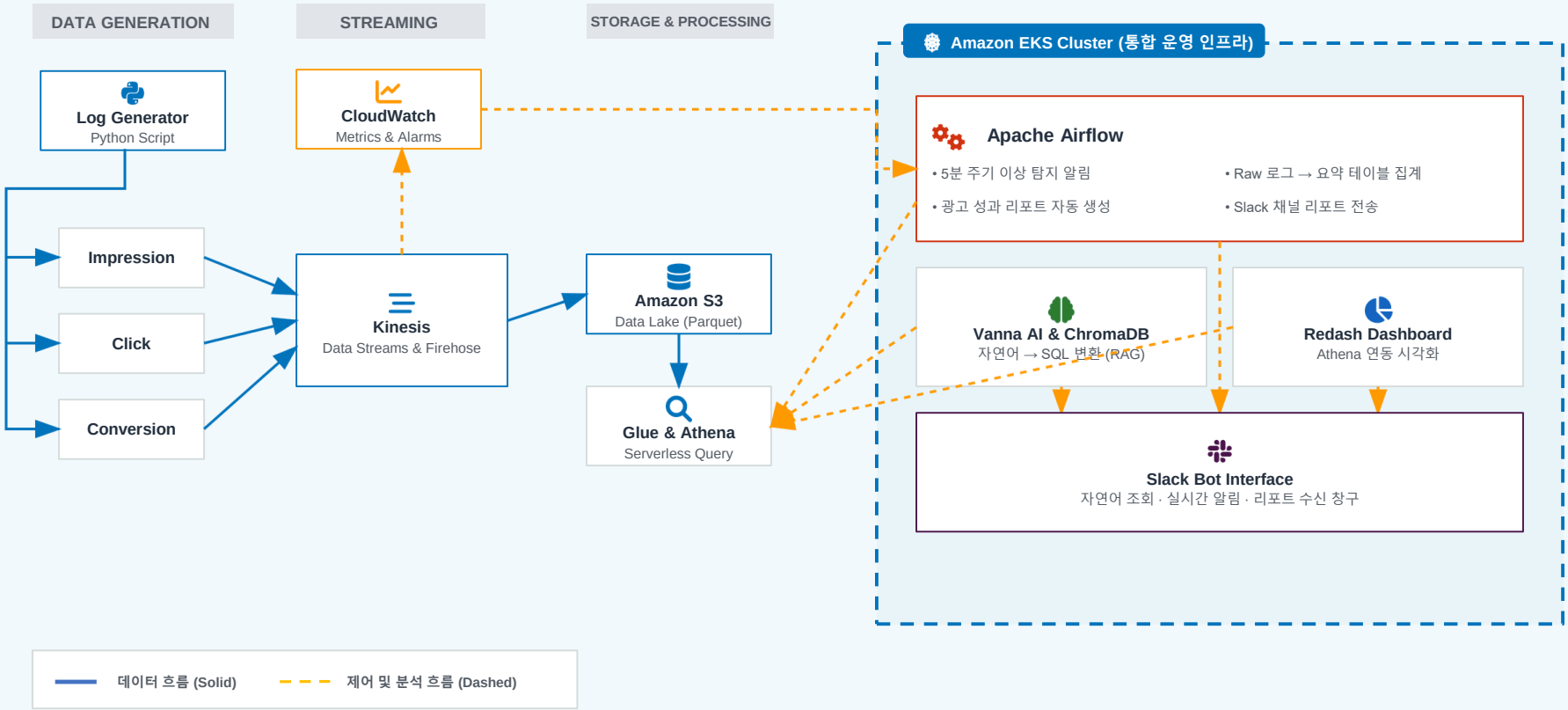
SECTION 03

전체 프로세스

시스템 아키텍처	데이터 수집 및 저장	Text-to-SQL 파이프라인	대시보드 구성	리포트 자동화	이상 탐지 알림	인프라 운영

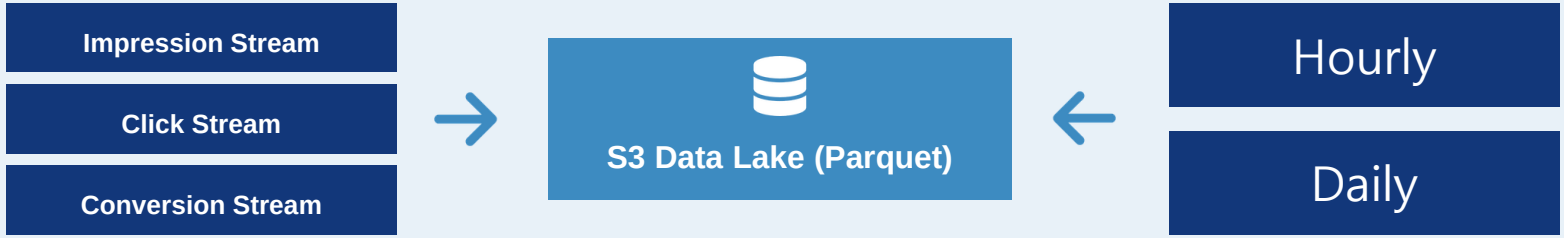
Engineering

시스템 아키텍처 및 광고 데이터 흐름



데이터 수집 및 저장 (Kinesis + S3/Athena)

"트래픽·타이밍이 완전히 다른 3가지를 독립 스트림으로 분리"



파티셔닝

- year/month/day/hour 구조로 분할
- 단일 날짜 조회 시 전체 스캔 방지

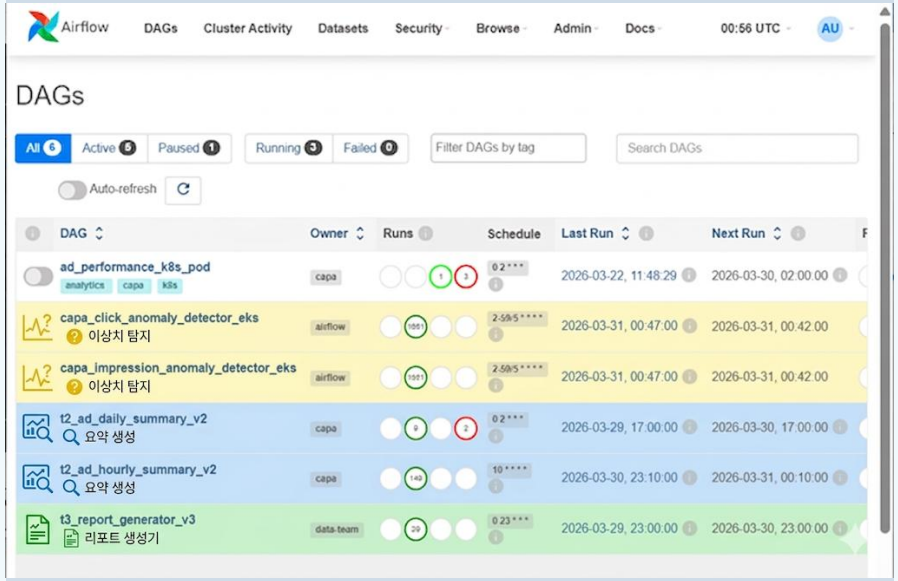
Parquet 형식

- 컬럼 기반 압축 저장
- Athena 스캔 비용 획기적 절감

Airflow Summary 테이블

- 1시간/1일 단위 사전 집계
- 대시보드·LLM은 가벼운 요약 연산만 수행

Airflow DAG 파이프라인



파이프라인 자동화

"분석 워크플로우 엔진: 복잡한 데이터 가공 및 분석 공정의 완전 자동화"

KubernetesPodOperator 도입

Pandas, Matplotlib 등 무거운 라이브러리 의존성을 격리하여 워커 노드 부하를 방지하고 리소스를 효율화합니다.

이상치 탐지, 리포트 자동화, 요약 테이블 생성 DAG

"독립적으로 가동되는 각 DAG의 Airflow 기반 일괄 주기 제어 및 상태 모니터링"

대시보드 구축 — Why Redash?

Why Redash?

쿼리 기반 시각화

Athena 쿼리를 그대로 붙여넣어 즉시 대시보드화 가능
.

파라미터 지원

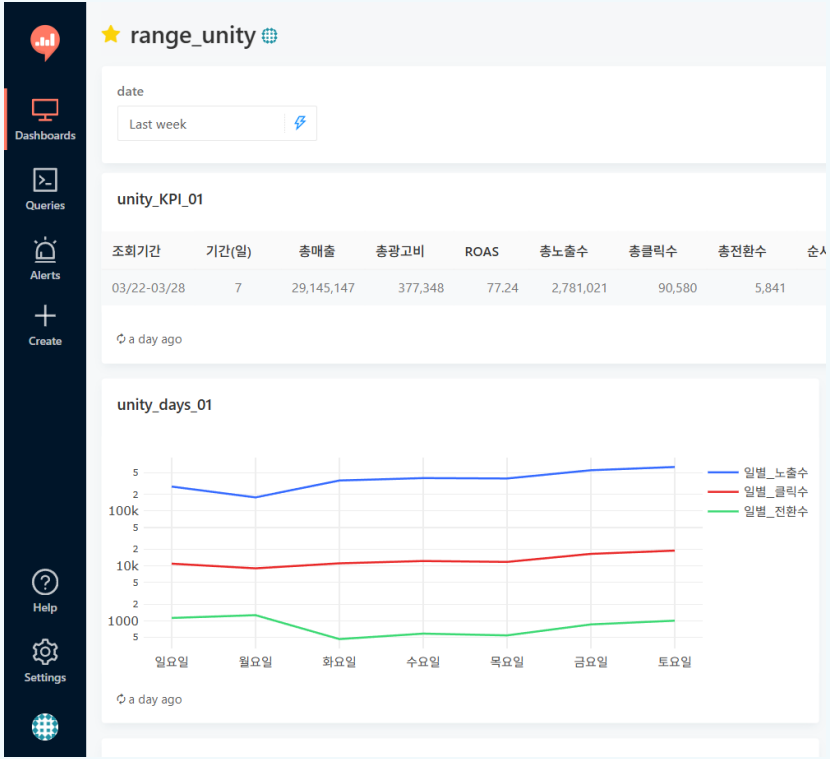
쿼리 파라미터 {{ 변수명 }} Jinja2 문법으로
날짜 조건을 마케터가 직접 변경 가능

쿼리 결과 일원화

AWS Athena 접근을 Redash로 일원화하여
Redash DB에서 쿼리 결과를 한곳에서 관리가 가능

사용자 접근

사용자가 Redash 계정을 통해 직접 접속해,
생성된 쿼리를 수정하고, 대시보드를 생성할 수 있음.



03 전체 프로세스

어떻게 자연어가 SQL이 되는가 — 11단계 Text-to-SQL 파이프라인

일반 LLM

없는 테이블·컬럼명을 지어냅니다 (할루시네이션)

CAPA

사내 스키마를 학습 → RAG로 맥락 주입 → 11단계 검증

입력 처리	RAG 검색	SQL 생성/검증	실행/분석	기록
Step 0 Conversation Retriever	Step 4 RAG Retriever	Step 5 SQL Generator	Step 7~9 Query Execution	Step 11 History Recorder
Step 1 Intent Classifier		Step 6 SQL Validator	Step 10 AI Analyzer	
Step 2 Question Refiner				
Step 3 Keyword Extractor				

멀티턴: DynamoDB → Step 0 조회 → 5턴 이력 → 프롬프트 주입 → 후속 질문 처리

비동기: 쿼리 수십 초 → task_id 즉시 반환 → 폴링 → 완료 시 결과 전달

정확도를 높이기 위해 고민한 것들

4중 검증·CoT·RAG 시딩으로 SQL 정확도를 구조적으로 끌어올리다

실행 전 4중 검증 및 가드레일

- **AST 지능형 파싱:** sqlglot으로 SELECT 외 명령 및 테이블 접근 원천 차단
- **Athena EXPLAIN 검증:** 실행 전 쿼리 플랜 무료 분석 → 문법 오류 및 풀스캔 사전 차단
- **SQL 가드레일:** 결과 행수 최대 1,000행 제한(LIMIT) 강제 적용
- **Self-Correction:** 검증 실패 시 에러 사유를 LLM에 피드백 → 최대 3회 자동 수정 루프

LLM의 '사고 절차' 프롬프트 설계

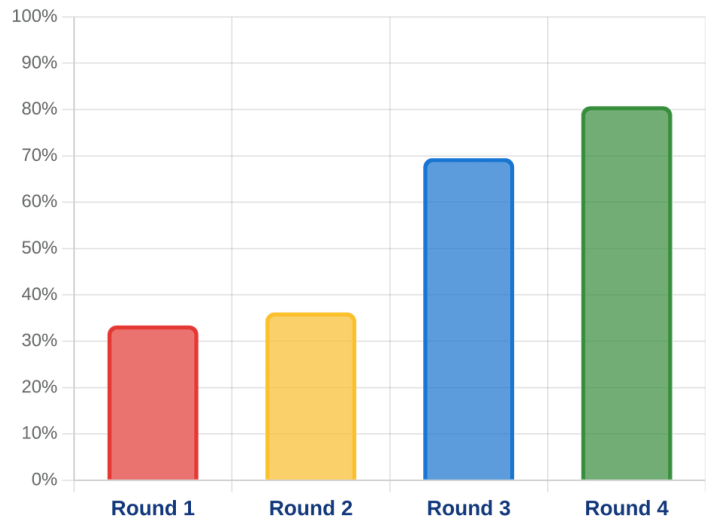
- **CoT(Chain of Thought):** DDL 확인 → 테이블 선택 → 파티션 변환 → 집계 설계 6단계 사고 절차 강제
- **프롬프트 인젝션 방지:** 시스템 지시문과 외부 데이터(RAG Context)를 별도 Content Block으로 완전 분리

RAG에 'Context 주입'

- **Athena 문법 최적화:** DATE(), BETWEEN 등 성능 저해 함수 차단 및 Presto 전용 구문 21종 지식화
- **비즈니스 로직 규격화:** CTR·CVR·ROAS 등 광고 지표 계산 공식 5종 사전 정의 및 시딩
- **Few-shot 기반 패턴 학습:** '어제', '지난주' 등 시간 표현 변환 70여 개 Golden QA 데이터셋 구축
- **오답 패턴 사전 보정:** 과거 실패 사례 8종을 역으로 학습시켜 hallucination 사전 억제

SQL 정확도 4라운드 개선 스토리

Exec (Execution Accuracy): 생성 SQL의 실행 결과값이 정답과 일치하는가



Round 1 (33.3%) — 기반 공사

QA 시딩 편향성 해결(21개) 및 날짜 동적 처리(Jinja2) 도입으로 기초 정확도 확보.

Round 2 (36.1%) — 사고 통제

CoT 6단계 프롬프트, Negative Rules, 계산식 공식화를 통해 생성 로직의 안정성 강화.

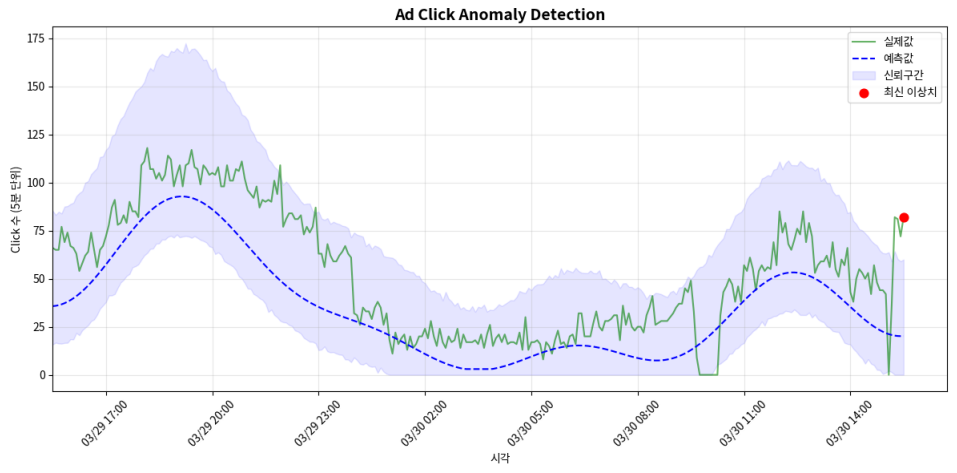
Round 3 (69.4%) — 검색 진화

한국어 특화 임베딩(ko-sroberta) 도입 및 Reranker 제거로 관련 지식 검색 효율 극대화.

Round 4 (80.6%) — 구조 돌파

QA 메타데이터 기반 DDL 동적 주입을 통해 테이블 오인식 한계를 극복하고 80%대 진입.

실시간 광고 이상 탐지 및 Slack 알림



앙상블 모델 적용

Prophet으로 트렌드 예측 후 Isolation Forest로 이상치를 교차 검증합니다.

즉각적인 Slack 알림

이상 징후 시 담당 채널로 즉시 알림을 발송합니다
.

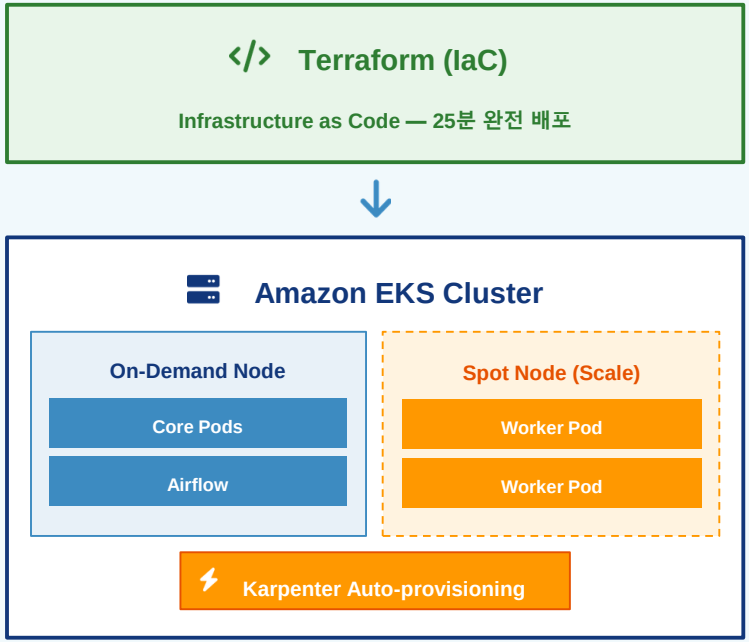
5분 주기 실시간 탐지

Airflow로 5분마다 최신 로그를 분석하여 즉시 탐지합니다.

예산 낭비 원천 차단

설정 오류나 비정상 트래픽으로 인한 광고비 소진을 방지합니다.

인프라 코드화 + 트래픽 대응 (Terraform + EKS + Karpenter)



Terraform (IaC) 기반 배포

인프라 전체를 코드로 관리하여 재현성을 확보하고, 클러스터 및 의존성 리소스를 25분 만에 완전 배포합니다.

Amazon EKS 오케스트레이션

안정적인 컨테이너 환경을 구축하여 마이크로서비스 아키텍처(MSA) 기반의 유연한 확장성을 제공합니다.

Karpenter 초고속 스케일링

트래픽 스파이크 발생 시 기존 Cluster Autoscaler 대비 압도적으로 빠른 속도로 노드를 프로비저닝합니다.

Spot 인스턴스 & 비용 최적화

Karpenter를 통해 저렴한 Spot 인스턴스를 적극 활용하고 유휴 리소스를 최소화하여 인프라 비용을 최대 70% 절감합니다.

CAPA 핵심 기술 스택 (Technical Spec)

AI & LLM

LLM

Claude 3.5 Haiku
Anthropic (고성능, 비용 효율적 모델)

엔진 / 히스토리

Vanna AI / DynamoDB
자연어-SQL 변환 및 세션 관리

벡터 DB

ChromaDB
오픈소스 기반 RAG 지식 베이스

데이터 파이프라인

스트리밍

Kinesis Data Streams
실시간 로그 수집 및 Firehose 연동

메타데이터

AWS Glue Catalog
크롤러 기반 스키마 자동 관리

저장소

Amazon S3
데이터 레이크 (Parquet 파티셔닝)

프로세싱 & 운영

플랫폼 / 인프라

Amazon EKS / Terraform
K8s 오케스트레이션 및 IaC

쿼리 엔진

Amazon Athena
서버리스 대용량 데이터 분산 쿼리

워크플로우

Apache Airflow
EKS 네이티브 기반 배치 자동화

확장성

Karpenter
노드 자동 스케일링 및 Spot 활용

인터페이스 & 모니터링

UI / BI

Slack / Redash
소켓 모드 봇 및 데이터 시각화

ML 엔진

Prophet / Isolation Forest
이상 탐지 앙상블 모델

모니터링

CloudWatch
실시간 지표 및 로그 추적

왜 이 기술인가? — 채택 근거

AI 지능화 및 데이터 파이프라인

카테고리	채택 기술	핵심 채택 사유 (도입 효과)
LLM	Claude Haiku	우수한 코드 작성 능력 및 높은 가성비
SQL 엔진	Vanna AI	Text-to-SQL 파이프라인 추상화 및 학습 용이성
Vector DB	ChromaDB	ko-sroberta 임베딩 모델 정합성 및 Vanna 호환성
이상 탐지	Prophet+I-Forest	계절성+통계 앙상블로 오탐 최소화
Streaming	Kinesis+Firehose	서버리스 기반 실시간 적재 및 Parquet 변환
Metadata	Glue Catalog	S3 데이터 스키마 변화 실시간 감지 및 자동화
쿼리 엔진	Amazon Athena	서버 운영 없는 즉시 쿼리 및 S3와 호환성
Storage	Amazon S3	파티셔닝 기반 대용량 통합 관리 및 저비용

클라우드 네이티브 및 운영 인프라

카테고리	채택 기술	핵심 채택 사유 (도입 효과)
플랫폼	Amazon EKS	Helm 기반 통합 관리 및 파드 단위 격리 & 확장성
오토스케일러	Karpenter	초고속 노드/파드 탄력적 관리 및 비용 절감
IaC	Terraform	인프라/K8s 리소스 단일 코드베이스 관리
History DB	DynamoDB	멀티턴 대화 이력 고속 조회 및 프리티어 활용
배치 관리	Apache Airflow	요약·이상탐지·리포트 주기적 스케줄링
BI 도구	Redash	Athena 연동 및 대시보드 & 쿼리 api 지원
인터페이스	Slack	챗봇 지원 및 스레드 기반 협업 최적화
모니터링	CloudWatch	Kinesis 스트림 모니터링해서 이상치 탐지를 위해 도입

SECTION 04

결론 및 회고

성과 비교
(AS-IS vs TO-BE)

시행착오 및 회고

향후 발전 방향

PROJECT

AS-IS vs TO-BE — 도입 전후 비교

AS-IS (과거)

데이터 조회 (Ask)
데이터팀에 요청 후 **수 시간~수 일 대기**. 의사결정 타이밍 지연.

정기 리포트 (Report)
매주/매월 담당자가 **수기 엑셀 다운로드 및 집계**. 반복적인 리소스 낭비.

이상 탐지 (Alert)
사람이 대시보드를 볼 때만 인지. **사후 발견으로 인한 예산 낭비** 발생.

대시보드 (Dashboard)
지표 확인마다 **매번 데이터팀 요청 반복**. 실시간 셀프 확인 불가.

TO-BE (현재)

데이터 조회 (Ask)
슬랙에서 자연어로 질문 시 **수십 초 내 즉시 답변 및 시각화**.

정기 리포트 (Report)
Airflow 기반 **완전 자동화**. 정해진 시간에 슬랙으로 자동 전송.

이상 탐지 (Alert)
AI 모델이 5분 주기로 실시간 감지. **이상 발생 즉시 슬랙 알림**.

대시보드 (Dashboard)
Redash 대시보드 **원클릭 셀프 확인**. SQL 없이 언제든 실시간 지표 조회.

데이터 추출 업무 소요 시간 99% 단축 (반나절 → 1분 이내)

시행착오 & 배운 점

인프라 & AI 엔진

EKS 노드 그룹 분리 실패

동일 노드 내 Airflow 워커의 과부하 작업으로 인한 자원 부족(oom: out of memory)으로 서버가 다운되는 현상

Karpenter를 도입하여 작업 특성별로 노드를 분리 프로비저닝하여 해결

Reranker 도입 후 롤백

Reranker 도입으로 정확도 개선을 기대했으나, 정확도는 오르지 않고 50초 이상의 지연 시간이 발생하여 UX 저하

사용되는 테이블이 2개 뿐이라 Reranker 도입은 오버엔지니어링. 롤백하고, ChromaDB의 메타데이터 필터링을 고도화하는 방향으로 선회

데이터 수집 & 시각화

데이터 파이프라인 연결방식

생성된 Raw데이터가 하나의 파이프로만 연결되었으나, 데이터 특성에 맞게 각각의 파이프라인으로 연결되어야 함.

노출/클릭/전환 3가지 데이터 특성에 맞는 개별 연결

Redash 거리감

관련 국내 자료가 부족하여, Redash에 진입장벽이 발생

인터페이스를 숙지하고, 임의의 대시보드를 수차례 생성

Redash 쿼리 파라미터

쿼리 파라미터 기능과 쿼리에서 설정

Redash에서 들어오는 쿼리 파라미터를 문자열로 취급

실시간 이상치 탐지

신뢰구간 상단 차등 가중치 부여

신뢰구간 상단이 비즈니스 맥락 없이 모델 예측값만으로 결정되고 있었음

데이터 특성별 가중치 차등적용 (I × 1.25, C × 1.5)

Prophet 신뢰구간 내 음수

트래픽이 적은 시간대에 신뢰구간 하한이 음수로 예측됨

하한을 음수가 아닌 '0'으로 보정하여 비현실적 상황 제거

CloudWatch 집계 처리시간

즉시 조회 시 집계가 완료되지 않아, 수집 시점에 따라 데이터가 누락될 수 있었음

2분의 버퍼를 두어 데이터를 안정적으로 수집

발전방향 (Future Work)

이상치 탐지 재학습 및 확장

데이터 실시간 변화를 반영하기 위한
주기적 모델 자동 재학습

Report 생성기 내 분석 기능 강화

AI가 Report 생성 전에 데이터를 분석하여
다각도의 인사이트를 도출하여 의사결정을 지원

역질문 봇 (Proactive Bot)

사용자의 질문을 기다리지 않고,
AI가 먼저 인사이트를 발견하여 제안

> *"ROAS가 떨어졌네요. 캠페인 A를 중단할까요?"*

피드백 루프 (Feedback Loop)

사용자의 피드백을 수집하여 SQL 품질을
지속적으로 개선

참고 자료 및 출처

- 데이터 접근성 -

> **[IAB/BWG State of Data 2026]** (Martech.org 인용)

(https://martech.org/75-of-marketers-say-their-measurement-systems-are-falling-short/ | 2026.02)

- 생산성 저하 -

> **[IAB/BWG State of Data 2026]** (Martech.org 인용)

https://martech.org/75-of-marketers-say-their-measurement-systems-are-falling-short/ | 2026.02)

- 국내 사례

> **[데이블]** (https://dabletech.oopy.io/2ce5bbc0-e5c2-8089-9dcd-c449b51eba46)

> **[SK Planet]** (https://tacademy.skplanet.com/front/centernews/viewCenterNews.do?seq=538#)

> **[우아한형제들]** (https://techblog.woowahan.com/18144/)

Q&A

경청해 주셔서 감사합니다